dame un ejemplo de algoritmo de punto medio

El **algoritmo de punto medio** (también llamado *Midpoint Algorithm*) se usa mucho en gráficos por computadora para dibujar figuras como líneas o circunferencias usando solo operaciones simples.

Voy a darte un ejemplo clásico: **dibujar una circunferencia con el algoritmo de punto medio**.

Ejemplo: Circunferencia con algoritmo de punto medio

Supongamos que queremos dibujar una circunferencia de:

- Centro: $(0, 0)$
- Radio: $r = 5$

Idea del algoritmo

En lugar de calcular toda la ecuación de la circunferencia:

$$x^2 + y^2 = r^2$$

El algoritmo decide en cada paso cuál píxel es mejor elegir usando un **punto medio** entre dos opciones posibles.

Fórmula de decisión inicial

El parámetro inicial es:

$$p_0 = 1 - r$$

Para este caso:

$$p_0 = 1 - 5 = -4$$

🔹 Reglas del algoritmo

En cada paso:

- Si $p < 0$ → elegimos el punto **Este (E)**

$$p_{k+1} = p_k + 2x + 3$$

- Si $p \geq 0$ → elegimos el punto **Sureste (SE)**

$$p_{k+1} = p_k + 2x - 2y + 5$$

🔹 Iteraciones

Paso	x	y	p	Decisión
0	0	5	-4	E
1	1	5	-1	E
2	2	5	4	SE
3	3	4	3	SE
4	4	3	6	SE

🔹 Puntos generados (primer octante)

$$(0,5), (1,5), (2,5), (3,4), (4,3)$$

Luego se reflejan para completar la circunferencia.

🧠 Pseudocódigo

```
Entrada: radio r, centro (xc, yc)

x = 0
y = r
p = 1 - r

Mientras x <= y:
```

```
    Dibujar puntos simétricos:
        (xc + x, yc + y)
        (xc - x, yc + y)
        (xc + x, yc - y)
        (xc - x, yc - y)
        (xc + y, yc + x)
        (xc - y, yc + x)
        (xc + y, yc - x)
        (xc - y, yc - x)

    x = x + 1

    Si p < 0:
        p = p + 2*x + 1
    Si no:
        y = y - 1
        p = p + 2*x - 2*y + 1
```

📌 ¿Dónde se usa?

- Gráficos por computadora 🎮
- Motores de renderizado
- Sistemas donde se requiere eficiencia (sin decimales)

Si quieres, puedo darte también el **algoritmo de punto medio para líneas (Bresenham)** o implementarlo en **C++, Python o Java paso a paso**.

que es el camelCast para variables? en html o js

El término correcto es **camelCase** (no *camelCast*), y es una forma de escribir nombres de variables muy usada en **JavaScript** y también en atributos relacionados con **HTML** cuando interactúan con JS.

🐫 ¿Qué es camelCase?

Es una convención donde:

- La **primera palabra va en minúscula**
- Cada palabra siguiente empieza con **mayúscula**
- No se usan espacios ni guiones

👉 Ejemplo:

```
let nombreUsuario = "Juan";
```

JavaScript

Se llama "camel" porque las mayúsculas parecen las jorobas de un camello 🐫.

🔹 En JavaScript

En JavaScript es la forma **más común y recomendada** para nombrar variables y funciones:

```
let numeroDeEstudiantes = 30;

function calcularPromedioNotas() {
```

JavaScript

✔️ Es el estándar en JS
✔️ Mejora la legibilidad

🔹 En HTML (cuando se usa con JS)

En HTML normalmente se usan guiones (-), pero cuando JavaScript accede a esos atributos, se transforman a camelCase.

👉 Ejemplo:

HTML

En JavaScript:

JavaScript

🔎 Observa:

- `data-user-name` → `userName` (camelCase en JS)

⚠️ Comparación rápida

Estilo	Ejemplo	Uso común
camelCase	`miVariableEjemplo`	JavaScript
snake_case	`mi_variable`	Python
kebab-case	`mi-variable`	HTML / CSS

📌 Resumen

- **camelCase** es una forma de nombrar variables sin espacios
- Muy usada en **JavaScript**
- En HTML aparece indirectamente cuando JS accede a atributos

---

Si quieres, puedo revisar tu código y decirte si estás usando bien camelCase o ayudarte a convertir variables 👍

tengo un error en alguna de estas partes y no se a que se deba
/**

```
 * UMNG - Facultad de Ingeniería
 * Asignatura: Introducción a la Computación Gráfica
 * Estudiante: Mauricio Enrique Alvarino Mercado
 * * Este código debe ser estructurado de forma modular.
 */

// Única función autorizada para dibujar
function plotPixel(ctx, x, y, color = "#1a1a1a") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

/**
 * Retorna los centros donde se ubicarán los polígonos
 * @param {number} r - Radio de la órbita
 * @param {number} n - Cantidad de polígonos
 * @returns {Array} [{x, y}, ...]
 */
function getOrbitalPositions(r, n) {
    // Implementar lógica de distribución circular
}

/**
 * Algoritmo de Bresenham para líneas
 */
function bresenhamLine(x0, y0, x1, y1, color) {
    x0 = Math.round(x0);
    y0 = Math.round(y0);
    x1 = Math.round(x1);
    y1 = Math.round(y1);
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

     while (true) {
        // Pintar el píxel actual
        plotPixel(ctx, x0, y0, color);
```

```
        // Condición de parada
        if (x0 === x1 && y0 === y1) break;

        let p2 = 2 * p;

        if (p2 > -dy) {
            p -= dy;
            x0 += sx;
        }

        if (p2 < dx) {
            p += dx;
            y0 += sy;
        }
}

/**
 * Algoritmo de Punto Medio para circunferencias
 * Dibujar una circunferencia utilizando el metodo del
punto medio aprovechando la simetria de 8 ectantes,
haciendo todo el circulo calculando solo el primer ectante
 *
 * @param {CanvasRenderingContext2D} ctx - Contexto 2D
del canvas
 * @param {number} centerX - Coordenada X del centro
de la circunferencia
 * @param {number} centerY - Coordenada Y del centro de
la circunferencia
 * @param {number} r - Radio de la circunferencia en
píxeles
 * @param {string} [color="#ffffff"] - Color de la
circunferencia
 */

function midpointCircle(centerX, centerY, r, color) {
    // Implementación obligatoria por el estudiante
    let x = 0;
    let y = r;

    let p = 1 - r;
```

```
    // Dibujar los 8 ectantes en el primer paso
    plotPixel(ctx, centerX + x, centerY + y, color);
    plotPixel(ctx, centerX - x, centerY + y, color);
    plotPixel(ctx, centerX + x, centerY - y, color);
    plotPixel(ctx, centerX - x, centerY - y, color);
    plotPixel(ctx, centerX + y, centerY + x, color);
    plotPixel(ctx, centerX - y, centerY + x, color);
    plotPixel(ctx, centerX + y, centerY - x, color);
    plotPixel(ctx, centerX - y, centerY - x, color);

    while (x < y) {
        x++;
        if (p < 0) {
            //Y se mantiene igual pero se actualiza el punto de
decisión
            p += 2 * x + 1;
        } else{
            //Y se decrementa y se actualiza el punto de
decisión a partir de la nueva coordenada
            y--;
            p += 2 * (x - y) + 1;
        }
        //Con cada nuevo punto calculado se dibujan los 8
ectantes correspondientes
        plotPixel(ctx, centerX + x, centerY + y, color);
        plotPixel(ctx, centerX - x, centerY + y, color);
        plotPixel(ctx, centerX + x, centerY - y, color);
        plotPixel(ctx, centerX - x, centerY - y, color);
        plotPixel(ctx, centerX + y, centerY + x, color);
        plotPixel(ctx, centerX - y, centerY + x, color);
        plotPixel(ctx, centerX + y, centerY - x, color);
        plotPixel(ctx, centerX - y, centerY - x, color);
    }
}
```

Hay varios errores claros en tu código (no es uno solo). Te los señalo directo y con solución, porque algunos rompen completamente la ejecución.

❌ 1. Error en Bresenham (el más grave)

Dentro de `bresenhamLine` usas una variable que **no existe**:

JavaScript

🔴 Problema:

- Nunca declaraste `p`
- La variable correcta es `err`

✅ Corrección

Cambia TODO ese bloque por esto:

```javascript
function bresenhamLine(x0, y0, x1, y1, color) {
    x0 = Math.round(x0);
    y0 = Math.round(y0);
    x1 = Math.round(x1);
    y1 = Math.round(y1);

    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

    while (true) {
        plotPixel(ctx, x0, y0, color);

        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * err;

        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }

        if (e2 < dx) {
```

```
            err += dx;
            y0 += sy;
        }
```

JavaScript

❌ 2. Falta cerrar la función `bresenhamLine`

Tu función abre `{` pero **nunca la cierras** antes del siguiente comentario.

🔴 Eso rompe TODO el script.

✅ Solución

Asegúrate de cerrar con:

JavaScript

Justo después del `while`.

❌ 3. Uso de `ctx` sin pasarlo como parámetro

En varias funciones haces esto:

JavaScript

Pero tu función NO recibe `ctx`:

JavaScript

🔴 Problema

`ctx` puede ser:

- undefined ❌
- global (mala práctica ⚠️)

✅ Solución correcta (modular como pide tu profe)

JavaScript

Y lo mismo aquí:

JavaScript

❌ 4. `getOrbitalPositions` está vacío

```
function getOrbitalPositions(r, n) {
```

JavaScript

Si la estás usando, te va a devolver `undefined`.

✅ Ejemplo correcto

```
function getOrbitalPositions(r, n) {
    let positions = [];

    for (let i = 0; i < n; i++) {
        let angle = (2 * Math.PI * i) / n;

        let x = r * Math.cos(angle);
        let y = r * Math.sin(angle);
```

```
        positions.push({ x, y });
    }
```

JavaScript

⚠️ 5. Detalles menores

✔️ Ortografía técnica (no rompe código, pero ojo)

- "ectantes" → **octantes**

✅ RESUMEN RÁPIDO

Tus errores principales son:

1. ❌ Usar `p` en vez de `err` en Bresenham
2. ❌ No cerrar la función
3. ❌ No pasar `ctx` como parámetro
4. ❌ Función vacía (`getOrbitalPositions`)

💡 Recomendación (importante para tu materia)

Tu profesor dijo:

> "código estructurado de forma modular"

Entonces SIEMPRE:

JavaScript

Nada de variables globales.

Si quieres, en el siguiente mensaje puedo:

- ↪ Revisarte TODO el proyecto completo
- O ↪ ayudarte a integrar línea + circunferencia + órbitas en un solo sistema listo para entregar 🚀

```
/**
 * Retorna los centros donde se ubicarán los polígonos
 * @param {number} r - Radio de la órbita
 * @param {number} n - Cantidad de polígonos
 * @returns {Array} [{x, y}, ...]
 */
function getOrbitalPositions(r, n) {
    // Implementar lógica de distribución circular

}
```

necesito implementar una funcion de esta forma, pero no logro captar la idea, sera que me puedes ayudar con esto, no puedo usar metodos de ctx ni nada por el estilo y pues manejo las otras funciones que ya te mostre en un mensaje anterior

Vas bien, esto es más sencillo de lo que parece—solo que hay que verlo geométricamente.

🧠 Idea clave (lo que debes entender)

Quieres ubicar **n puntos distribuidos uniformemente en una circunferencia** de radio `r`.

👉 Eso se hace usando ángulos.

Un círculo completo tiene:

$$2\pi \text{ radianes}$$

Entonces divides ese círculo en `n` partes:

$$\text{ángulo} = \frac{2\pi}{n}$$

Y cada punto se calcula con:

$$x = r \cdot \cos(\theta)$$

$$y = r \cdot \sin(\theta)$$

✅ Implementación correcta

```javascript
function getOrbitalPositions(r, n) {
    let positions = [];

    for (let i = 0; i < n; i++) {
        let angle = (2 * Math.PI * i) / n;

        let x = r * Math.cos(angle);
        let y = r * Math.sin(angle);

        positions.push({ x: x, y: y });
    }
```

JavaScript

🔍 ¿Qué está pasando aquí?

- `i` → número del punto (0, 1, 2, ...)
- `angle` → en qué parte del círculo estás
- `cos` → coordenada horizontal
- `sin` → coordenada vertical

📌 Ejemplo práctico

JavaScript

Te da algo como:

```
[
  {x: 100, y: 0},
  {x: 0, y: 100},
  {x: -100, y: 0},
```

JavaScript

👉 Es básicamente una cruz (4 puntos bien distribuidos)

⚠️ MUY IMPORTANTE (detalle típico de error)

Estos puntos están centrados en **(0,0)**

Si quieres moverlos a otro centro (por ejemplo `(cx, cy)`), haces:

JavaScript

🎯 Cómo se conecta con tu proyecto

Tú ya tienes:

- `plotPixel`
- `bresenhamLine`
- `midpointCircle`

👉 Entonces esta función solo calcula **dónde van los centros**
(no dibuja nada, como te pidieron ✔️)

💡 Bonus (para que te quede perfecto)

Si luego dibujas polígonos:

```javascript
let centros = getOrbitalPositions(150, 6);

centros.forEach(p => {
```

JavaScript

Si quieres, te puedo ayudar a:

- ↳ conectar esto con tus líneas (Bresenham)
- o ↳ armar el sistema completo tipo "órbitas + polígonos" como proyecto final 👌

pero necesito tener en cuenta el centro de la orbita y un angulo de desplazamiento

Perfecto—eso ya es la versión "completa" del problema 👍
Ahora no solo distribuyes puntos, sino que:

1. **Desplazas el centro** de la órbita
2. **Rotas toda la distribución** con un ángulo inicial

🧠 Idea ajustada

Antes tenías:

$$x = r \cdot \cos(\theta), \quad y = r \cdot \sin(\theta)$$

Ahora agregas:

- Centro: (c_x, c_y)
- Desplazamiento angular: θ_0

Entonces:

$$x = c_x + r \cdot \cos(\theta + \theta_0)$$

$$y = c_y + r \cdot \sin(\theta + \theta_0)$$

✅ Implementación (la que necesitas)

```javascript
function getOrbitalPositions(r, n, cx, cy, angleOffset = 0) {
    let positions = [];

    for (let i = 0; i < n; i++) {
        let angle = (2 * Math.PI * i) / n + angleOffset;

        let x = cx + r * Math.cos(angle);
        let y = cy + r * Math.sin(angle);

        positions.push({ x: x, y: y });
    }
```

JavaScript

🔍 Parámetros

- `r` → radio de la órbita
- `n` → cantidad de puntos
- `cx, cy` → centro de la órbita
- `angleOffset` → rotación inicial (en radianes)

📌 Ejemplo

JavaScript

👉 Esto:

- centra la órbita en (300,300)
- rota todo 45°

🎯 Qué hace el ángulo de desplazamiento

Sin offset:

```
(100,0) → derecha
(0,100) → arriba
```

Con offset (π/4):

```
todo gira en diagonal 🔄
```

⚠️ Error típico (muy común)

Si usas grados:

JavaScript

Debes convertir a radianes:

JavaScript

💡 Conexión con tu proyecto

Esto te permite:

- Rotar animaciones 🔄
- Crear órbitas inclinadas
- Alinear polígonos con orientación específica

Si quieres, en el siguiente paso puedo ayudarte a:

- ↳ hacer que los polígonos **también roten sobre sí mismos**
- o ↳ animar toda la órbita en tiempo real (queda muy pro para entregar) 🚀

y si mi orbita no inicia en 0°?

Buena pregunta—y en realidad ya lo estás resolviendo con el **angleOffset**, solo que hay que interpretarlo bien.

🧠 Caso: la órbita NO inicia en 0°

Normalmente el primer punto está en:

- 0° → a la derecha del centro

Si quieres que empiece en otro ángulo (por ejemplo 90°, 30°, etc.), simplemente **cambias el ángulo inicial**.

✅ La idea correcta

En vez de:

$$\theta = \frac{2\pi i}{n}$$

usas:

$$\theta = \frac{2\pi i}{n} + \theta_{\text{inicio}}$$

👉 Ese θ_{inicio} es exactamente lo que necesitas.

✅ Implementación (igual que antes, pero entendida correctamente)

```
function getOrbitalPositions(r, n, cx, cy, startAngle = 0) {
    let positions = [];
```

```javascript
for (let i = 0; i < n; i++) {
    let angle = (2 * Math.PI * i) / n + startAngle;

    let x = cx + r * Math.cos(angle);
    let y = cy + r * Math.sin(angle);

    positions.push({ x, y });
}
```

JavaScript

📌 Ejemplos claros

🔹 Iniciar en 0° (por defecto)

JavaScript

Primer punto → derecha

🔹 Iniciar en 90°

JavaScript

Primer punto → arriba

🔹 Iniciar en 180°

JavaScript

Primer punto → izquierda

🔹 **Iniciar en 45°**

JavaScript

Primer punto → diagonal

⚠️ IMPORTANTE (te puede salvar la nota)

JavaScript usa **radianes**, no grados:

$$\text{radianes} = \text{grados} \cdot \frac{\pi}{180}$$

👉 Ejemplo:

JavaScript

🎯 Resumen claro

- Sí puedes iniciar en cualquier ángulo ✅
- Solo necesitas un **offset angular** ✅
- Eso rota TODA la órbita, no solo un punto ✅

Si quieres, puedo ir un paso más allá y ayudarte a:

- ↳ hacer que la órbita **gire en el tiempo (animación)**
- o ↳ alinear los polígonos con la tangente de la órbita (nivel pro 👀)

<!DOCTYPE html>
<html lang="en">

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Sistema de Dispersión Geométrica Orbital
(Rasterización Manual)
    <style>
        body {
            background-color: #1e1e2e;
            font-family: Arial, sans-serif;
            color: #cdd6f4;
            margin: 0;
            padding: 20px;
        }
        canvas {
            border: 1px solid #202024;
            background-color: #a3a3b65b;
            border-radius: 8px;
            box-shadow: 0 4px 20px rgba(0, 0, 0, 0.5);
        }
        .container {
            display: flex;
            justify-content: center;
            gap: 20px;
            margin-bottom: 15px;

        }
        input {
            width: 60px;
            margin: 5px;
        }
    </style>
</title>
</head>
<body>
     <div style="text-align:center;"class="container">

            <h1>Sistema de Dispersión Geométrica
Orbital</h1>
            <p style="color:#555; font-
```

size:14px;">Rasterización Manual — Algoritmos de Bresenham y Punto Medio</p>

<canvas id="orbitalCanvas" width="400" height="400"></canvas>

Radio Orbital (R): —
Polígonos (N): —
Lados por polígono (k): —
<button id="btnRegenerar" onclick="iniciarSistema()">↺ Regenerar Sistema</button>

<script src="script.js"></script>
</body>
</html>

este es mi html y pues no veo que inicie un canvas para que se grafique lo que se tiene que graficar

Tu problema no es el canvas en sí — el `<canvas>` **sí está creado correctamente**.
El error real está en el HTML y en cómo estás inicializando todo.

Te explico directo 👇

❌ 1. Error en el `<title>` (rompe el DOM)

Tienes esto:

HTML

```
<style>
```

🔴 Problema:

- **No cerraste el** `<title>`
- El navegador interpreta mal todo lo que sigue

✅ Solución

HTML

❌ 2. Nunca inicializas el canvas en JS

El HTML solo crea el canvas, pero **no lo estás usando** hasta que hagas esto en tu `script.js`:

JavaScript

Si eso no está, nada se dibuja.

❌ 3. No llamas a ninguna función al cargar la página

Solo tienes esto:

HTML

Pero eso solo funciona si haces clic.

✅ Solución (muy importante)

Agrega esto en tu `script.js` :

```
window.onload = function () {
```

JavaScript

❌ 4. Puede que `iniciarSistema()` ni exista

Si no tienes esta función en tu JS:

```
function iniciarSistema() {
```

JavaScript

👉 entonces el botón tampoco hará nada

❌ 5. Error silencioso típico

Si en tu JS hay errores (como el de Bresenham que tenías), el script **se detiene** y no dibuja nada.

👉 Abre la consola del navegador:

- F12 → pestaña **Console**

✅ Ejemplo mínimo funcional

Pon esto en tu `script.js` para probar:

```
const canvas = document.getElementById("orbitalCanvas");
const ctx = canvas.getContext("2d");

function iniciarSistema() {
```

```
    // prueba simple
    ctx.fillStyle = "red";
    ctx.fillRect(200, 200, 5, 5);
`
```

JavaScript

👉 Si ves un punto rojo → el canvas funciona ✔️

🎯 Resumen (lo que te está fallando)

1. ❌ `<title>` mal cerrado
2. ❌ No inicializas `ctx`
3. ❌ No ejecutas nada al cargar
4. ❌ Posibles errores JS bloqueando todo

💡 Siguiente paso

Si quieres, mándame tu `script.js` completo y te lo dejo:

- ✔️ totalmente funcional
- ✔️ con órbitas
- ✔️ con polígonos
- ✔️ listo para entregar

Porque ya estás a nada de tener todo armado 🚀

segun entiendo no puedo usar ctx, que mi funcion plot es la que se debe encargar de eso

Vas bien, pero hay una confusión importante 👇

👉 **Sí puedes usar** `ctx`, lo que NO puedes hacer es:

- usar `ctx.lineTo`, `ctx.arc`, `ctx.stroke`, etc. ❌

- porque debes rasterizar “a mano”

👉 **Pero** `ctx` **sigue siendo necesario** para:

- pintar píxeles (`fillRect`)
- y eso lo haces a través de tu función `plotPixel` ✔️

🧠 Entonces, ¿cómo funciona correctamente?

Tu arquitectura (que es la correcta) es:

```
ctx → plotPixel → algoritmos (Bresenham, punto medio, etc.)
```

✅ Forma correcta de trabajar

1. `plotPixel` (único lugar donde usas ctx)

```javascript
function plotPixel(ctx, x, y, color = "#1a1a1a") {
    ctx.fillStyle = color;
```

✔️ Aquí SÍ usas `ctx`

✔️ Aquí SÍ puedes usar `fillRect`

---

2. Tus algoritmos (NO usan métodos de ctx directamente)

Pero **sí reciben** `ctx` :

```javascript
function bresenhamLine(ctx, x0, y0, x1, y1, color) {
```

👉 No llamas `ctx.lineTo` , pero sí pasas `ctx` a `plotPixel`

3. Inicialización (obligatoria)

JavaScript

4. Función principal

```
function iniciarSistema() {
    // ejemplo
```

JavaScript

5. Ejecutar al cargar

JavaScript

🚨 ERROR típico (probablemente el tuyo)

Si haces esto:

```
function bresenhamLine(x0, y0, x1, y1, color) {
```

JavaScript

🔴 `ctx` no está definido dentro de la función

→ no dibuja nada

✅ Solución correcta

Siempre pasa `ctx` :

JavaScript

🎯 Resumen claro

- ✔ NO puedes usar funciones de dibujo avanzadas de canvas
- ✔ SÍ puedes usar `ctx.fillRect` (a través de `plotPixel`)
- ✔ `ctx` debe pasarse como parámetro (código modular)
- ✔ Si no pasas `ctx` , no se dibuja nada

💡 Traducción a lo que te pidió el profe

> "Rasterización manual"

Significa:

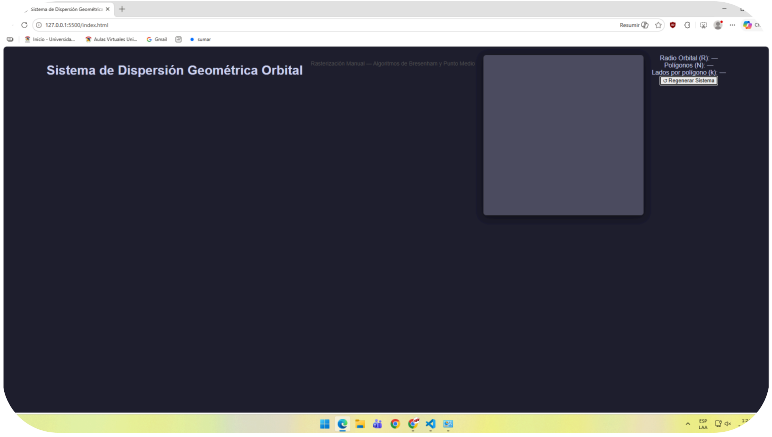
- Tú decides qué píxeles dibujar ✔
- No usas primitivas del canvas ❌

Pero **igual necesitas el canvas como "pantalla"**

Si quieres, mándame tu `script.js` y te lo dejo funcionando completo con:

- ↳ órbita
- ↳ polígonos
- ↳ Bresenham
- ↳ punto medio

y listo para entregar 🔥



no me gusta que se vea asi, quiero que se vea centradas las cosas y organizadas

Sí, lo que tienes ahora se ve desbalanceado porque todo está en un solo contenedor flex sin una estructura clara. Vamos a organizarlo bien para que quede **centrado, limpio y profesional** (tipo entrega final 🔥).

🎯 OBJETIVO

- Canvas centrado
- Panel de información a un lado
- Título arriba centrado
- Todo alineado y con buen spacing

✅ HTML CORREGIDO (estructura limpia)

Reemplaza tu `<body>` por esto:

```
<body>
    <h1 class="title">Sistema de Dispersión Geométrica Orbital</h1>
    <p class="subtitle">Rasterización Manual — Algoritmos de Bresenham y Punto Me

    <div class="main-container">
```

```html
        <div class="canvas-container">
            <canvas id="orbitalCanvas" width="400" height="400"></canvas>
        </div>

        <div class="panel">
            <div id="info-r">Radio Orbital (R): —</div>
            <div id="info-n">Polígonos (N): —</div>
            <div id="info-k">Lados por polígono (k): —</div>

            <button onclick="iniciarSistema()">↺ Regenerar Sistema</button>
        </div>

    </div>
```

HTML

🎨 CSS MEJORADO (esto es lo importante)

Reemplaza tu `<style>` por este:

```css
body {
    background-color: #1e1e2e;
    font-family: Arial, sans-serif;
    color: #cdd6f4;
    margin: 0;
    padding: 20px;
    text-align: center;
}

.title {
    margin-bottom: 5px;
}

.subtitle {
    color: #888;
    font-size: 14px;
    margin-bottom: 30px;
}

.main-container {
    display: flex;
    justify-content: center;
```

```
    align-items: center;
    gap: 40px;
}

/* Canvas */
.canvas-container {
    background-color: #2a2a3d;
    padding: 15px;
    border-radius: 10px;
    box-shadow: 0 4px 20px rgba(0,0,0,0.5);
}

canvas {
    background-color: #a3a3b65b;
    border-radius: 8px;
}

/* Panel lateral */
.panel {
    display: flex;
    flex-direction: column;
    gap: 10px;
    background-color: #2a2a3d;
    padding: 20px;
    border-radius: 10px;
    min-width: 200px;
    text-align: left;
}

.panel button {
    margin-top: 10px;
    padding: 8px;
    background-color: #89b4fa;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}

.panel button:hover {
```

CSS

🔥 RESULTADO

Con esto vas a tener:

- ✔ Título centrado arriba
- ✔ Canvas centrado
- ✔ Panel organizado a la derecha
- ✔ Espaciado limpio
- ✔ Diseño tipo aplicación real

💡 OPCIONAL (nivel más pro)

Si quieres que TODO quede perfectamente centrado en la pantalla:

```
body {
    height: 100vh;
    display: flex;
    flex-direction: column;
```

CSS

🚀 Si quieres subir nivel

Te puedo ayudar a dejarlo tipo:

- interfaz tipo simulador
- animación de órbitas
- sliders para cambiar R, N, k en tiempo real

Solo dime 👀

/**
* UMNG - Facultad de Ingeniería
* Asignatura: Introducción a la Computación Gráfica
* Estudiante: Mauricio Enrique Alvarino Mercado
* * Este código debe ser estructurado de forma modular.
*/

```
// Única función autorizada para dibujar
const canvas =
document.getElementById("orbitalCanvas");
const ctx = canvas.getContext("2d");

function plotPixel(ctx=canvas.getContext("2d"), x=30,
y=125, color = "#1a1a1a") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

/**
 * Retorna los centros donde se ubicarán los polígonos
 * @param {number} r - Radio de la órbita
 * @param {number} n - Cantidad de polígonos
 * @param {number} cx - Coordenada X del centro de la
órbita
 * @param {number} cy - Coordenada Y del centro de la
órbita
 * @param {number} OffA - Ángulo de desplazamiento
para la animación
 * @returns {Array} [{x, y}, ...]
 */
//Añadi cx, cy y OffA para tener los centros de la órbita, el
ángulo de desplazamiento para la animación y así poder
reutilizar esta función para calcular las posiciones en cada
frame
function getOrbitalPositions(cx, cy, r, n, OffA) {
    // Implementar lógica de distribución circular
    const positions = [];

    // Paso angular entre polígono y polígono en la
circunferencia, calculado dividiendo 360 grados (2 * PI
radianes) entre la cantidad de polígonos
    const pasoAngular = (2 * Math.PI) / n;

    // Distribuir los centros de los polígonos en una
circunferencia de radio r alrededor del centro del canvas
    for (let i = 0; i < n; i++) {
        // Ángulo correspondiente al i-ésimo polígono
```

```
        const angulo = OffA + i * pasoAngular;

        // Proyección trigonométrica sobre la circunferencia
        const x = cx + r * Math.cos(angulo);
        const y = cy + r * Math.sin(angulo);

        // Almacenar el centro calculado como objeto {x, y}
        positions.push({ x, y });
    }
    return positions;
}

/**
 * Algoritmo de Bresenham para líneas
 */
function bresenhamLine(x0, y0, x1, y1, color) {
    x0 = Math.round(x0);
    y0 = Math.round(y0);
    x1 = Math.round(x1);
    y1 = Math.round(y1);
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

     while (true) {
        // Pintar el píxel actual
        plotPixel(ctx, x0, y0, color);

        // Condición de parada
        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * err;

        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }
```

```
        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}

/**
 * Algoritmo de Punto Medio para circunferencias
 * Dibujar una circunferencia utilizando el metodo del
punto medio aprovechando la simetria de 8 ectantes,
haciendo todo el circulo calculando solo el primer ectante
 *
 * @param {CanvasRenderingContext2D} ctx - Contexto 2D
del canvas
 * @param {number} centerX - Coordenada X del centro
de la circunferencia
 * @param {number} centerY - Coordenada Y del centro de
la circunferencia
 * @param {number} r - Radio de la circunferencia en
píxeles
 * @param {string} [color="#ffffff"] - Color de la
circunferencia
 */

function midpointCircle(centerX, centerY, r, color) {
    // Implementación obligatoria por el estudiante

    let x = 0;
    let y = r;

    let p = 1 - r;



    // Dibujar los 8 ectantes en el primer paso
    plotPixel(ctx, centerX + x, centerY + y, color);
    plotPixel(ctx, centerX - x, centerY + y, color);
    plotPixel(ctx, centerX + x, centerY - y, color);
    plotPixel(ctx, centerX - x, centerY - y, color);
    plotPixel(ctx, centerX + y, centerY + x, color);
```

```
    plotPixel(ctx, centerX - y, centerY + x, color);
    plotPixel(ctx, centerX + y, centerY - x, color);
    plotPixel(ctx, centerX - y, centerY - x, color);

    while (x < y) {
        x++;
        if (p < 0) {
            //Y se mantiene igual pero se actualiza el punto de
decisión
            p += 2 * x + 1;
        } else{
            //Y se decrementa y se actualiza el punto de
decisión a partir de la nueva coordenada
            y--;
            p += 2 * (x - y) + 1;
        }
        //Con cada nuevo punto calculado se dibujan los 8
ectantes correspondientes
        plotPixel(ctx, centerX + x, centerY + y, color);
        plotPixel(ctx, centerX - x, centerY + y, color);
        plotPixel(ctx, centerX + x, centerY - y, color);
        plotPixel(ctx, centerX - x, centerY - y, color);
        plotPixel(ctx, centerX + y, centerY + x, color);
        plotPixel(ctx, centerX - y, centerY + x, color);
        plotPixel(ctx, centerX + y, centerY - x, color);
        plotPixel(ctx, centerX - y, centerY - x, color);
    }
}

/**
 * Calcula y retorna los vértices de un polígono regular de
k lados,
 * con centro en (cx, cy), radio interno radioP y rotado
según angleOffset.
 *
 * Fórmula para el i-ésimo vértice:
 *   ángulo_i = OffA + i * (2pi / k)
 *   xi = cx + radioP * cos(angulo_i)
 *   yi = cy + radioP * sin(angulo_i)
 *
```

```
 * @param {number} cx - Coordenada X del centro del polígono
 * @param {number} cy - Coordenada Y del centro del polígono
 * @param {number} radioP - Radio circunscrito del polígono (distancia centro-vértice)
 * @param {number} k - Número de lados (y vértices) del polígono regular
 * @param {number} [OffA] - Rotación inicial del polígono en radianes
 * @returns {Array<{x: number, y: number}>} Arreglo de vértices en orden
 */
function getPolygonVertices(cx, cy, radioP, k, OffA = 0) {
    const vertices = [];

    // Paso angular entre vértices consecutivos: 360° / k
    const pasoAngular = (2 * Math.PI) / k;

    for (let i = 0; i < k; i++) {
        const angulo = OffA + i * pasoAngular;

        const x = cx + radioP * Math.cos(angulo);
        const y = cy + radioP * Math.sin(angulo);

        vertices.push({ x, y });
    }

    // Retornar el arreglo de vértices antes de dibujar para permitir su reutilización en otras funciones
    return vertices;
}

/**
 * Dibuja un polígono regular usando los vértices retornados por getPolygonVertices.
 * Conecta cada par de vértices consecutivos con el Algoritmo de Bresenham.
 * El último vértice se conecta de vuelta al primero para cerrar la figura.
```

```
 *
 * @param {CanvasRenderingContext2D} ctx - Contexto 2D
del canvas
 * @param {number} cx - Coordenada X del centro
 * @param {number} cy - Coordenada Y del centro
 * @param {number} radioP - Radio circunscrito en píxeles
 * @param {number} k - Número de lados
 * @param {number} OffA - Rotación inicial en radianes
 * @param {string} color - Color del polígono
 */
function drawPolygon(ctx, cx, cy, radioP, k, OffA, color) {
    // Obtener arreglo de vértices {x, y} antes de dibujar
    const vertices = getPolygonVertices(cx, cy, radioP, k,
OffA);

    // Conectar cada vértice con el siguiente usando
Bresenham
    for (let i = 0; i < vertices.length; i++) {
        const actual = vertices[i];
        // El módulo cierra el polígono, el último vértice
conecta al primero
        const siguiente = vertices[(i + 1) % vertices.length];

        bresenhamLine(ctx, actual.x, actual.y, siguiente.x,
siguiente.y, color);
    }
}
window.onload = function () {
    iniciarSistema();
};
```

tengo todo eso y ya quiero que comience a funcionar en el canva (pasa que cambie los valores del plot para ver si funcionaba y ya vi que no funciona)

Estás **muy cerca**, pero hay un par de errores clave que hacen que *no se dibuje absolutamente nada*. Te los marco directo y cómo arreglarlos 👇

❌ 1. Estás llamando mal a `bresenhamLine`

Definiste:

JavaScript

Pero lo llamas así:

JavaScript

🔴 Problema:

- Estás pasando `ctx` como si fuera `x0`
- Todo queda desplazado → no se ve nada

✅ SOLUCIÓN

Opción correcta (recomendada por modularidad):

JavaScript

Y dentro:

JavaScript

❌ 2. Nunca defines `iniciarSistema()`

Tienes:

```
window.onload = function () {
```

JavaScript

🔴 Pero esa función NO existe → error silencioso → no dibuja nada

---

✅ SOLUCIÓN (mínimo para probar)

Agrega esto:

```javascript
function iniciarSistema() {
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    // centro del canvas
    let cx = canvas.width / 2;
    let cy = canvas.height / 2;

    // dibujar órbita
    midpointCircle(cx, cy, 120, "white");

    // obtener posiciones
    let posiciones = getOrbitalPositions(cx, cy, 120, 6, 0);

    // dibujar polígonos
    posiciones.forEach(p => {
        drawPolygon(ctx, p.x, p.y, 30, 5, 0, "red");
```

JavaScript

❌ 3. `plotPixel` está mal definido (esto sí te puede romper cosas)

Tienes:

JavaScript

🔴 Problema:

- Estás creando un `ctx` nuevo cada vez ❌

- Default innecesario ❌

✅ Déjalo así (correcto):

```javascript
function plotPixel(ctx, x, y, color = "#1a1a1a") {
    ctx.fillStyle = color;
```

JavaScript

❌ 4. Falta limpiar el canvas (si pruebas varias veces)

Siempre haz:

JavaScript

✅ CHECKLIST FINAL (lo que debe quedar)

- ✔ `plotPixel(ctx, ...)` sin valores por defecto raros
- ✔ `bresenhamLine(ctx, ...)` recibiendo ctx
- ✔ `drawPolygon` llamando bien a Bresenham
- ✔ `iniciarSistema()` EXISTE
- ✔ `window.onload` lo ejecuta

🎯 SI TODO ESTÁ BIEN...

Deberías ver:

- Un círculo (órbita)
- Polígonos distribuidos alrededor

💥 Si aún no aparece nada

Haz esto:

1. Presiona **F12**
2. Ve a **Console**
3. Mira si hay errores (seguro hay uno)

🚀 Si quieres siguiente nivel

Te puedo ayudar a:

- ↳ animar la órbita (OffA cambiando en el tiempo)
- ↳ hacer que los polígonos roten
- ↳ sliders interactivos (R, N, k)

Tu código ya está a **90% de ser proyecto final pro** 👀

me salen estos errores Uncaught TypeError: ctx.fillRect is not a function
at plotPixel (script.js:14:9)
at midpointCircle (script.js:111:5)
at iniciarSistema (script.js:212:5)
at window.onload (script.js:224:5)

Ese error no es aleatorio—te está diciendo exactamente qué está mal:

```
ctx.fillRect is not a function
```

👉 Traducción: **lo que estás pasando como `ctx` NO es un contexto 2D** (es otra cosa, probablemente un número).

💥 EL PROBLEMA REAL (clave)

Tienes esta inconsistencia:

🔴 Definición actual

JavaScript

🔴 Pero la llamas así:

JavaScript

👉 Entonces dentro de la función:

JavaScript

Y luego:

JavaScript

👉 O sea:

JavaScript

Y en algún punto `ctx` deja de ser el contexto → boom 💥 error.

✅ SOLUCIÓN (la correcta y limpia)

1. Corrige la firma de Bresenham

JavaScript

---

2. Déjala así completa

```javascript
function bresenhamLine(ctx, x0, y0, x1, y1, color) {
    x0 = Math.round(x0);
    y0 = Math.round(y0);
    x1 = Math.round(x1);
    y1 = Math.round(y1);

    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

    while (true) {
        plotPixel(ctx, x0, y0, color);

        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * err;

        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }

        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
```

JavaScript

---

3. Verifica `plotPixel`

Debe estar EXACTAMENTE así:

```javascript
function plotPixel(ctx, x, y, color = "#1a1a1a") {
    ctx.fillStyle = color;
```

JavaScript

```
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}
```

❌ SIN valores por defecto raros
❌ SIN redefinir ctx dentro

---

4. Asegúrate que `ctx` es válido

JavaScript

🔍 DEBUG RÁPIDO (haz esto)

Antes de dibujar:

JavaScript

👉 Debe mostrar algo como:

```
CanvasRenderingContext2D
```

Si ves:

- `undefined` ❌
- número ❌
- objeto raro ❌

→ ahí está el error

🎯 RESUMEN

Tu error viene de:

✔ Pasar mal los parámetros en Bresenham
✔ `ctx` deja de ser contexto y se convierte en otra cosa

✔ `fillRect` deja de existir → error

🚀 BONUS (para evitar estos bugs siempre)

Haz TODAS tus funciones así:

JavaScript

👉 y siempre llama:

JavaScript

Si quieres, mándame tu `iniciarSistema()` y te dejo todo funcionando con:

- órbita
- polígonos
- animación

ya estás literalmente a un paso de que todo se vea 🔥

```
/**
 * UMNG - Facultad de Ingeniería
 * Asignatura: Introducción a la Computación Gráfica
 * Estudiante: Mauricio Enrique Alvarino Mercado
 * * Este código debe ser estructurado de forma modular.
 */

// Única función autorizada para dibujar
const canvas =
document.getElementById("orbitalCanvas");
const ctx = canvas.getContext("2d");
```

```
function plotPixel(ctx, x, y, color = "#1a1a1a") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

/**
 * Retorna los centros donde se ubicarán los polígonos
 * @param {number} r - Radio de la órbita
 * @param {number} n - Cantidad de polígonos
 * @param {number} cx - Coordenada X del centro de la
órbita
 * @param {number} cy - Coordenada Y del centro de la
órbita
 * @param {number} OffA - Ángulo de desplazamiento
para la animación
 * @returns {Array} [{x, y}, ...]
 */
//Añadi cx, cy y OffA para tener los centros de la órbita, el
ángulo de desplazamiento para la animación y así poder
reutilizar esta función para calcular las posiciones en cada
frame
function getOrbitalPositions(cx, cy, r, n, OffA) {
    // Implementar lógica de distribución circular
    const positions = [];

    // Paso angular entre polígono y polígono en la
circunferencia, calculado dividiendo 360 grados (2 * PI
radianes) entre la cantidad de polígonos
    const pasoAngular = (2 * Math.PI) / n;

    // Distribuir los centros de los polígonos en una
circunferencia de radio r alrededor del centro del canvas
    for (let i = 0; i < n; i++) {
        // Ángulo correspondiente al i-ésimo polígono
        const angulo = OffA + i * pasoAngular;

        // Proyección trigonométrica sobre la circunferencia
        const x = cx + r * Math.cos(angulo);
        const y = cy + r * Math.sin(angulo);
```

```
        // Almacenar el centro calculado como objeto {x, y}
        positions.push({ x, y });
    }
    return positions;
}

/**
 * Algoritmo de Bresenham para líneas
 * @param {CanvasRenderingContext2D} ctx - Contexto 2D
del canvas
 * @param {number} x0 - Coordenada X del punto inicial
 * @param {number} y0 - Coordenada Y del punto inicial
 * @param {number} x1 - Coordenada X del punto final
 * @param {number} y1 - Coordenada Y del punto final
 * @param {string} [color="#ffffff"] - Color del segmento
 */
function bresenhamLine(ctx, x0, y0, x1, y1, color="#ffffff") {
    x0 = Math.round(x0);
    y0 = Math.round(y0);
    x1 = Math.round(x1);
    y1 = Math.round(y1);
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;
    let err = dx - dy;

     while (true) {
        // Pintar el píxel actual
        plotPixel(ctx, x0, y0, color);

        // Condición de parada
        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * err;

        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }
```

```
        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}

/**
 * Algoritmo de Punto Medio para circunferencias
 * Dibujar una circunferencia utilizando el metodo del
punto medio aprovechando la simetria de 8 ectantes,
haciendo todo el circulo calculando solo el primer ectante
 *
 * @param {CanvasRenderingContext2D} ctx - Contexto 2D
del canvas
 * @param {number} centerX - Coordenada X del centro
de la circunferencia
 * @param {number} centerY - Coordenada Y del centro de
la circunferencia
 * @param {number} r - Radio de la circunferencia en
píxeles
 * @param {string} [color="#ffffff"] - Color de la
circunferencia
 */

function midpointCircle(ctx, centerX, centerY, r,
color="#ffffff") {
    // Implementación obligatoria por el estudiante

    let x = 0;
    let y = r;

    let p = 1 - r;



    // Dibujar los 8 ectantes en el primer paso
    plotPixel(ctx, centerX + x, centerY + y, color);
    plotPixel(ctx, centerX - x, centerY + y, color);
    plotPixel(ctx, centerX + x, centerY - y, color);
```

```
    plotPixel(ctx, centerX - x, centerY - y, color);
    plotPixel(ctx, centerX + y, centerY + x, color);
    plotPixel(ctx, centerX - y, centerY + x, color);
    plotPixel(ctx, centerX + y, centerY - x, color);
    plotPixel(ctx, centerX - y, centerY - x, color);

    while (x < y) {
        x++;
        if (p < 0) {
            //Y se mantiene igual pero se actualiza el punto de
decisión
            p += 2 * x + 1;
        } else{
            //Y se decrementa y se actualiza el punto de
decisión a partir de la nueva coordenada
            y--;
            p += 2 * (x - y) + 1;
        }
        //Con cada nuevo punto calculado se dibujan los 8
ectantes correspondientes
        plotPixel(ctx, centerX + x, centerY + y, color);
        plotPixel(ctx, centerX - x, centerY + y, color);
        plotPixel(ctx, centerX + x, centerY - y, color);
        plotPixel(ctx, centerX - x, centerY - y, color);
        plotPixel(ctx, centerX + y, centerY + x, color);
        plotPixel(ctx, centerX - y, centerY + x, color);
        plotPixel(ctx, centerX + y, centerY - x, color);
        plotPixel(ctx, centerX - y, centerY - x, color);
    }
}

/**
 * Calcula y retorna los vértices de un polígono regular de
k lados,
 * con centro en (cx, cy), radio interno radioP y rotado
según angleOffset.
 *
 * Fórmula para el i-ésimo vértice:
 *   ángulo_i = OffA + i * (2pi / k)
 *   xi = cx + radioP * cos(angulo_i)
```

```
 *   yi = cy + radioP * sin(angulo_i)
 *
 * @param {number} cx - Coordenada X del centro del polígono
 * @param {number} cy - Coordenada Y del centro del polígono
 * @param {number} radioP - Radio circunscrito del polígono (distancia centro-vértice)
 * @param {number} k - Número de lados (y vértices) del polígono regular
 * @param {number} [OffA] - Rotación inicial del polígono en radianes
 * @returns {Array<{x: number, y: number}>} Arreglo de vértices en orden
 */
function getPolygonVertices(cx, cy, radioP, k, OffA = 0) {
    const vertices = [];

    // Paso angular entre vértices consecutivos: 360° / k
    const pasoAngular = (2 * Math.PI) / k;

    for (let i = 0; i < k; i++) {
        const angulo = OffA + i * pasoAngular;

        const x = cx + radioP * Math.cos(angulo);
        const y = cy + radioP * Math.sin(angulo);

        vertices.push({ x, y });
    }

    // Retornar el arreglo de vértices antes de dibujar para permitir su reutilización en otras funciones
    return vertices;
}

/**
 * Dibuja un polígono regular usando los vértices retornados por getPolygonVertices.
 * Conecta cada par de vértices consecutivos con el Algoritmo de Bresenham.
```

```
 * El último vértice se conecta de vuelta al primero para
cerrar la figura.
 *
 * @param {CanvasRenderingContext2D} ctx - Contexto 2D
del canvas
 * @param {number} cx - Coordenada X del centro
 * @param {number} cy - Coordenada Y del centro
 * @param {number} radioP - Radio circunscrito en píxeles
 * @param {number} k - Número de lados
 * @param {number} OffA - Rotación inicial en radianes
 * @param {string} color - Color del polígono
 */
function drawPolygon(ctx, cx, cy, radioP, k, OffA, color) {
    // Obtener arreglo de vértices {x, y} antes de dibujar
    const vertices = getPolygonVertices(cx, cy, radioP, k,
OffA);

    // Conectar cada vértice con el siguiente usando
Bresenham
    for (let i = 0; i < vertices.length; i++) {
        const actual = vertices[i];
        // El módulo cierra el polígono, el último vértice
conecta al primero
        const siguiente = vertices[(i + 1) % vertices.length];

        bresenhamLine(ctx, actual.x, actual.y, siguiente.x,
siguiente.y, color);
    }
}

function iniciarSistema() {
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    // centro del canvas
    let cx = canvas.width / 2;
    let cy = canvas.height / 2;

    // dibujar órbita
    midpointCircle(cx, cy, 120, "white");
```

```
    // obtener posiciones
    let posiciones = getOrbitalPositions(cx, cy, 120, 6, 0);

    // dibujar polígonos
    posiciones.forEach(p => {
        drawPolygon(ctx, p.x, p.y, 30, 5, 0, "red");
    });
}

window.onload = function () {
    iniciarSistema();
};
```

lo tengo todo asi y me salen esos errores

Ya casi lo tienes—el error ahora es **muy específico y fácil de arreglar** 👇

💥 ERROR ACTUAL

En `iniciarSistema()` estás llamando mal a `midpointCircle`:

JavaScript

🔴 Problema

Tu función está definida así:

JavaScript

👉 El **primer parámetro es** `ctx`

Pero tú le estás pasando:

JavaScript

Entonces dentro de `plotPixel` :

JavaScript

👉 `ctx` ahora es un número → 💥 `fillRect is not a function`

✅ SOLUCIÓN

Corrige la llamada:

JavaScript

🎯 POR QUÉ PASA ESTO

Estás haciendo esto sin darte cuenta:

JavaScript

Se interpreta como:

```
ctx = cx
centerX = cy
```

JavaScript

👉 TODO queda corrido → error seguro

✅ TU `iniciarSistema` CORREGIDO

```javascript
function iniciarSistema() {
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    let cx = canvas.width / 2;
    let cy = canvas.height / 2;

    // ✓ CORREGIDO
    midpointCircle(ctx, cx, cy, 120, "white");

    let posiciones = getOrbitalPositions(cx, cy, 120, 6, 0);

    posiciones.forEach(p => {
        drawPolygon(ctx, p.x, p.y, 30, 5, 0, "red");
```

JavaScript

🔥 RESUMEN (esto era TODO el problema)

✔️ Tu código está bien estructurado
✔️ Tus algoritmos están bien
❌ Solo estabas pasando mal `ctx` en UNA función

🚀 RESULTADO ESPERADO

Ahora deberías ver:

- 🟣 Un círculo (órbita)
- 🔺 Polígonos rojos alrededor

💡 TIP FINAL (para evitar esto SIEMPRE)

Haz una regla mental:

```
TODAS las funciones de dibujo empiezan con ctx
```

Ejemplo:

JavaScript

Si quieres, el siguiente paso natural es 🔥:

- ↳ animar la órbita con `OffA += 0.01`
- ↳ hacer que los polígonos roten también

y ya tienes proyecto de nivel alto 👀

bueno ahora necesito lo ultimo, necesito que los parametros se generen aleatoreamente, para que cada vez que se le de al boton de regenerar sistema pues salgan resultados diferentes

Perfecto, esto ya es el último paso lógico de tu proyecto 👌
Vamos a hacerlo bien: **parámetros aleatorios + actualización del panel + regeneración con botón**.

🎯 ¿Qué necesitas?

Cada vez que llames `iniciarSistema()`:

- Radio orbital (R) → aleatorio
- Número de polígonos (N) → aleatorio
- Lados por polígono (k) → aleatorio
- (opcional) rotación → aleatoria

✅ 1. Función para números aleatorios

```
function randomInt(min, max) {
```

JavaScript

✅ 2. Modificar `iniciarSistema()`

Reemplaza tu función por esta:

```
function iniciarSistema() {
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    let cx = canvas.width / 2;
    let cy = canvas.height / 2;

    // 🎲 PARÁMETROS ALEATORIOS
    let R = randomInt(80, 150);     // radio orbital
    let N = randomInt(3, 10);       // cantidad de polígonos
    let k = randomInt(3, 8);        // lados del polígono
    let offset = Math.random() * 2 * Math.PI; // rotación aleatoria

    // 🖥️ Mostrar en el panel
    document.getElementById("info-r").textContent = `Radio Orbital (R): ${R}`;
    document.getElementById("info-n").textContent = `Polígonos (N): ${N}`;
    document.getElementById("info-k").textContent = `Lados por polígono (k): ${k}

    // 🟣 Dibujar órbita
    midpointCircle(ctx, cx, cy, R, "white");

    // 📍 Obtener posiciones
    let posiciones = getOrbitalPositions(cx, cy, R, N, offset);

    // 🔺 Dibujar polígonos
    posiciones.forEach(p => {
        drawPolygon(ctx, p.x, p.y, 25, k, offset, "red");
```

JavaScript

✅ 3. Tu botón ya funciona

Ya tienes esto en HTML:

HTML

👉 Con eso basta ✔️

🔥 Resultado

Cada vez que:

- cargas la página
- haces clic en el botón

👉 Obtienes un sistema completamente distinto:

- diferente cantidad de polígonos
- diferentes formas
- diferente tamaño
- diferente orientación

💡 OPCIONAL (nivel pro)

Si quieres hacerlo más interesante:

🎨 Color aleatorio

```
function randomColor() {
```

JavaScript

Y en el dibujo:

JavaScript

🚀 Si quieres subir aún más nivel

Puedes agregar:

- animación orbital (rotación continua)
- sliders para controlar R, N, k
- velocidad de rotación

Si quieres, ↳ te armo la versión animada tipo simulador (queda brutal para entregar 👀)

necesito que me expliques paso a paso cada una pues no las entiendo, y si puedes hacerlo como tengo las explicaciones en mi codigo mejor

Perfecto, te lo explico **como lo vienes documentando en tu código**, paso a paso y sin saltos. La idea es que puedas copiar esto como comentarios y entender exactamente qué hace cada cosa.

🧩 1. Función `randomInt(min, max)`

```
/**
 * Genera un número entero aleatorio entre un mínimo y un máximo (incluidos).
 *
 * Fórmula:
 *   Math.random() → número decimal entre 0 y 1
 *   (max - min + 1) → tamaño del rango
 *   + min → desplaza el rango al valor mínimo deseado
 *
 * Ejemplo:
 *   randomInt(3, 6) puede devolver: 3, 4, 5 o 6
 *
 * @param {number} min - Valor mínimo
 * @param {number} max - Valor máximo
 * @returns {number} Número entero aleatorio en el rango [min, max]
 */
function randomInt(min, max) {
```

JavaScript

🎲 2. Generación de parámetros aleatorios

Esto va dentro de `iniciarSistema()`

```javascript
// Generar un radio orbital aleatorio entre 80 y 150 píxeles
let R = randomInt(80, 150);

// Generar cantidad de polígonos entre 3 y 10
let N = randomInt(3, 10);

// Generar cantidad de lados del polígono entre 3 y 8
let k = randomInt(3, 8);
```

JavaScript

🧠 Explicación clara

🔹 Radio (R)

JavaScript

- Controla el tamaño de la órbita
- No lo haces muy pequeño ni muy grande para que quepa en el canvas

---

🔹 Número de polígonos (N)

JavaScript

- Define cuántas figuras habrá en la órbita

- Mínimo 3 → para que se vea distribución circular
- Máximo 10 → para que no se sature

🔹 Lados del polígono (k)

JavaScript

- 3 → triángulo
- 4 → cuadrado
- 5+ → polígonos más complejos

ChatGPT Obtener Plus ×

🔹 Ángulo de desplazamiento (offset)

JavaScript

Paso a paso:

1. `Math.random()` → número entre 0 y 1
2. `* 2π` → lo conviertes a un ángulo completo
3. Resultado → cualquier orientación posible

👉 Esto hace que la órbita "gire" diferente cada vez

🖥️ 3. Mostrar los valores en pantalla

```
document.getElementById("info-r").textContent = `Radio Orbital (R): ${R}`;
```

JavaScript

🧠 ¿Qué hace esto?

- Busca el elemento del HTML por su `id`
- Cambia el texto que muestra
- `${R}` inserta el valor dentro del string

🟣 4. Dibujar la órbita

JavaScript

👉 Usa tu algoritmo de punto medio
👉 Dibuja la circunferencia base

📍 5. Calcular posiciones orbitales

JavaScript

🧠 ¿Qué hace esto?

- Divide el círculo en `N` partes
- Aplica el ángulo `offset`
- Devuelve posiciones `{x, y}`

🔺 6. Dibujar los polígonos

```
posiciones.forEach(p => {
```

JavaScript

🧠 Paso a paso

1. Recorre cada posición
2. Para cada punto:
   - usa `p.x`, `p.y` como centro
   - dibuja un polígono
   - con `k` lados
   - rotado con `offset`

🎯 FUNCIÓN COMPLETA DOCUMENTADA

Aquí la tienes como deberías dejarla:

```javascript
function iniciarSistema() {
    // Limpiar el canvas antes de dibujar un nuevo sistema
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    // Calcular el centro del canvas
    let cx = canvas.width / 2;
    let cy = canvas.height / 2;

    // ================================
    // GENERACIÓN DE PARÁMETROS ALEATORIOS
    // ================================

    // Radio orbital aleatorio
    let R = randomInt(80, 150);

    // Cantidad de polígonos
```

JavaScript

```javascript
    // Lados de cada polígono
    let k = randomInt(3, 8);

    // Ángulo de desplazamiento aleatorio
    let offset = Math.random() * 2 * Math.PI;

    // ================================
    // MOSTRAR INFORMACIÓN EN PANTALLA
    // ================================

    document.getElementById("info-r").textContent = `Radio Orbital (R): ${R}`;
```

```
    document.getElementById("info-n").textContent = `Polígonos (N): ${N}`;
    document.getElementById("info-k").textContent = `Lados por polígono (k): ${k}

    // ================================
    // DIBUJO DE LA ÓRBITA
    // ================================

    midpointCircle(ctx, cx, cy, R, "white");

    // ================================
    // CÁLCULO DE POSICIONES
    // ================================

    let posiciones = getOrbitalPositions(cx, cy, R, N, offset);

    // ================================
    // DIBUJO DE POLÍGONOS
    // ================================

    posiciones.forEach(p => {
        drawPolygon(ctx, p.x, p.y, 25, k, offset, "red");
    });
}
```

🚀 Con esto ya tienes

- ✔ Código modular
- ✔ Parámetros aleatorios
- ✔ Explicación clara tipo informe
- ✔ Listo para entregar

Si quieres, el siguiente paso (que te puede subir la nota bastante) es:

👉 ↳ **animar la órbita en tiempo real** (queda brutal)